

React JS Cheatsheet for Interview Preparation

For Freshers and Experienced Developers

A simple, detailed guide to explain React JS concepts in interviews with examples.

1. What is React JS and Why Use It?

What is it? React JS is a JavaScript library by Facebook for building interactive web interfaces. Think of it as LEGO blocks: you create small pieces (components) like buttons or forms and combine them to build apps.

Why Use It?

- **Fast:** Updates only changed parts of the page using Virtual DOM.
- **Reusable:** Build components once, use them anywhere.
- **Community:** Tons of tools (e.g., Redux, Next.js) and job opportunities.
- **Flexible:** Works for web, mobile (React Native), and server-side rendering.

Interview Tip: Say, “React is great for building fast, reusable UIs. It’s like LEGO—you make small components and snap them together.”

2. Virtual DOM

What is it? A lightweight copy of the real DOM (the web page’s structure) stored as a JavaScript object.

How It Works:

1. When something changes (e.g., clicking a button), React makes a new Virtual DOM.
2. It compares the new and old Virtual DOMs to find differences (called diffing).
3. Only the differences update the real DOM, making it fast.

Example: If a list has 100 items and one changes, React updates just that one item.

Interview Tip: Explain, “It’s like editing a draft document before updating the final version to save time.”

3. Reconciliation

What is it? The process where React updates the real DOM by comparing old and new component trees.

How It Works: React checks what changed (diffing) and applies minimal updates. Using key attributes in lists helps it know which items changed.

Example:

```
1 <ul>
2   <li key="1">Apple</li>
3   <li key="2">Banana</li>
4 </ul>
```

Adding a new item updates only that item, not the whole list.

Interview Tip: Say, “Reconciliation is like updating only the changed parts of a puzzle, not rebuilding it.”

4. React Fiber

What is it? React’s updated engine (since React 16) for smoother updates.

How It Works:

- Splits work into small chunks (fibers).
- Can pause low-priority tasks (e.g., background loading) to focus on urgent ones (e.g., button clicks).
- Makes apps feel faster, especially for animations.

Example: A button click updates instantly while a big list loads in the background.

Interview Tip: Explain, “Fiber is like a smart manager who prioritizes important tasks to keep the app responsive.”

5. State

What is it? Data that changes in a component, like a counter or form input, causing the UI to update.

Example:

```
1 import { useState } from 'react';
2 function Counter() {
3   const [count, setCount] = useState(0);
4   return (
5     <div>
6       <p>Count: {count}</p>
7       <button onClick={() => setCount(count + 1)}>Add</button>
8     </div>
9   );
10 }
```

Interview Tip: Say, “State is like a component’s memory—it holds data that can change and updates the UI.”

6. Props

What is it? Data passed from a parent component to a child, like passing notes.

Example:

```
1 function Greeting({ name }) {  
2   return <h1>Hello, {name}!</h1>;  
3 }  
4 function App() {  
5   return <Greeting name="Alice" />;  
6 }
```

Interview Tip: Explain, “Props are like instructions a parent gives to a child component, and they’re read-only.”

7. Props Drilling

What is it? Passing props through many nested components to reach a deep child.

Example:

```
1 function App() {  
2   return <Parent name="Alice" />;  
3 }  
4 function Parent({ name }) {  
5   return <Child name={name} />;  
6 }  
7 function Child({ name }) {  
8   return <h1>Hello, {name}!</h1>;  
9 }
```

Issue: Gets messy with many layers.

Interview Tip: Say, “It’s like passing a note through multiple people—Context or Redux avoids this.”

8. useState Hook

What is it? A way to add state to functional components.

How It Works: Gives you a state variable and a function to update it. Updates trigger re-renders.

Example:

```
1 import { useState } from 'react';  
2 function Toggle() {  
3   const [isOn, setIsOn] = useState(false);  
4   return (  
5     <button onClick={() => setIsOn(!isOn)}>  
6       {isOn ? 'On' : 'Off'}  
7     </button>  
8   );  
9 }
```

Interview Tip: Explain, “useState is like a light switch—it tracks state and flips the UI when it changes.”

9. useEffect Hook

What is it? A hook for side effects like fetching data or setting timers.

How It Works: Runs after rendering. Use a dependency array to control when it runs.

Example:

```
1 import { useState, useEffect } from 'react';
2 function DataFetcher() {
3   const [data, setData] = useState(null);
4   useEffect(() => {
5     fetch('https://api.example.com/data')
6       .then(res => res.json())
7       .then(setData);
8     return () => console.log('Cleanup');
9   }, []); // Runs once
10  return <div>{data ? data.name : 'Loading...'}</div>;
11 }
```

Interview Tip: Say, “useEffect is like setting up a task (e.g., fetching data) after the component appears.”

10. useRef Hook

What is it? A hook to hold values that don’t trigger re-renders, like a DOM reference.

How It Works: Returns an object with a `.current` property you can change.

Example:

```
1 import { useRef } from 'react';
2 function TextInput() {
3   const inputRef = useRef(null);
4   const focusInput = () => inputRef.current.focus();
5   return (
6     <div>
7       <input ref={inputRef} />
8       <button onClick={focusInput}>Focus</button>
9     </div>
10  );
11 }
```

Interview Tip: Explain, “useRef is like a sticky note you attach to a DOM element or value.”

11. useLayoutEffect Hook

What is it? Like `useEffect`, but runs before the browser paints the screen.

How It Works: Good for DOM measurements (e.g., element size).

Example:


```

1 import { useEffect, useRef } from 'react';
2 function Box() {
3   const divRef = useRef(null);
4   useEffect(() => {
5     const { height } = divRef.current.getBoundingClientRect();
6     console.log('Height:', height);
7   }, []);
8   return <div ref={divRef} style={{ height: '100px' }}>Box</div>;
9 }

```

Interview Tip: Say, “useLayoutEffect is like useEffect but runs earlier to adjust the UI before it shows.”

12. useReducer Hook

What is it? A hook for complex state logic, like a mini-Redux.

How It Works: Uses a reducer function to update state based on actions.

Example:

```

1 import { useReducer } from 'react';
2 const initialState = { count: 0 };
3 function reducer(state, action) {
4   switch (action.type) {
5     case 'increment': return { count: state.count + 1 };
6     case 'decrement': return { count: state.count - 1 };
7     default: return state;
8   }
9 }
10 function Counter() {
11   const [state, dispatch] = useReducer(reducer, initialState);
12   return (
13     <div>
14       <p>Count: {state.count}</p>
15       <button onClick={() => dispatch({ type: 'increment' })}>+</button>
16       <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
17     </div>
18   );
19 }

```

Interview Tip: Explain, “useReducer is like a traffic controller for complex state changes.”

13. useContext Hook

What is it? A hook to share data across components without props drilling.

Example:

```

1 import { createContext, useContext } from 'react';
2 const ThemeContext = createContext('light');
3 function App() {
4   return (
5     <ThemeContext.Provider value="dark">
6       <Display />

```

```

7     </ThemeContext.Provider>
8   );
9 }
10 function Display() {
11   const theme = useContext(ThemeContext);
12   return <div>Theme: {theme}</div>;
13 }

```

Interview Tip: Say, “useContext is like a shared bulletin board all components can read from.”

14. Lifecycle Methods with useEffect

What is it? In class components, methods like `componentDidMount` handle lifecycle events. In functional components, `useEffect` covers them.

How It Works:

- **Mount:** `useEffect(() => , [])` runs once after component mounts.
- **Update:** `useEffect(() => , [dep])` runs when dependencies change.
- **Unmount:** Return a cleanup function in `useEffect`.

Example:

```

1 import { useEffect } from 'react';
2 function Component() {
3   useEffect(() => {
4     console.log('Mounted');
5     return () => console.log('Unmounted');
6   }, []);
7   return <div>Hello</div>;
8 }

```

Interview Tip: Explain, “useEffect handles a component’s life stages, like birth, updates, and cleanup.”

15. useMemo Hook

What is it? A hook to cache expensive calculations to avoid repeating them.

How It Works: Only recalculates if dependencies change.

Example:

```

1 import { useMemo, useState } from 'react';
2 function ExpensiveComponent({ numbers }) {
3   const [count, setCount] = useState(0);
4   const sum = useMemo(() => {
5     console.log('Calculating...');
6     return numbers.reduce((a, b) => a + b, 0);
7   }, [numbers]);
8   return (
9     <div>
10       <p>Sum: {sum}</p>
11       <button onClick={() => setCount(count + 1)}>Re-render</button>

```

```
12     </div>
13   );
14 }
```

Interview Tip: Say, “useMemo is like saving a math result to avoid recalculating it.”

16. useCallback Hook

What is it? A hook to cache functions to avoid recreating them on re-renders.

How It Works: Only recreates the function if dependencies change.

Example:

```
1 import { useCallback, useState } from 'react';
2 function Button({ onClick }) {
3   return <button onClick={onClick}>Click</button>;
4 }
5 function App() {
6   const [count, setCount] = useState(0);
7   const handleClick = useCallback(() => setCount(count + 1), [count]);
8   return <Button onClick={handleClick} />;
9 }
```

Interview Tip: Explain, “useCallback is like saving a recipe so you don’t rewrite it every time.”

17. Optimizing Performance in React

How to Optimize:

- **useMemo:** Cache calculations.
- **useCallback:** Cache functions.
- **React.memo:** Prevent re-renders of unchanged components.
- **Lazy Loading:** Use `React.lazy` for big components.
- **Keys:** Use keys in lists to avoid re-rendering unchanged items.

Example:

```
1 import { memo, useMemo, useCallback } from 'react';
2 const Child = memo(({ data, onClick }) => {
3   console.log('Child rendered');
4   return <button onClick={onClick}>{data}</button>;
5 });
6 function App() {
7   const [count, setCount] = useState(0);
8   const data = useMemo(() => 'Hello', []);
9   const handleClick = useCallback(() => setCount(count + 1), [count]);
10  return <Child data={data} onClick={handleClick} />;
11 }
```

Interview Tip: Say, “Optimization is like tuning a car—useMemo and useCallback make it run efficiently.”

18. Pros and Cons of React JS

Pros:

- Fast updates with Virtual DOM.
- Reusable components save time.
- Huge ecosystem (React Native, Next.js).
- Flexible with other tools.

Cons:

- Takes time to learn (especially hooks).
- Needs extra libraries for routing or state management.
- JSX can feel strange at first.

Interview Tip: Say, “React is powerful and flexible but needs extra tools for big apps.”

19. Redux and Global State

What is Redux? A library to manage app-wide data (global state) in one place.

What is Global State? Data shared across components, like a user’s login status.

How Redux Works:

- **Store:** One place for all app data.
- **Actions:** Instructions to change data (e.g., { type: 'ADD' }).
- **Reducers:** Rules to update the store based on actions.

Interview Tip: Explain, “Redux is like a central bank—everyone accesses the same account.”

20. Global State vs. Props Drilling

Global State (Redux/Context):

- Shares data without passing through components.
- Cleaner for big apps.

Props Drilling:

- Passes data through every component layer.
- Okay for small apps, messy for big ones.

Example (Context vs. Drilling):

```
1 import { createContext, useContext } from 'react';
2 const UserContext = createContext(null);
3 function App() {
4   return (
5     <UserContext.Provider value="Alice">
6       <Child />
```



```

7     </UserContext.Provider>
8   );
9 }
10 function Child() {
11   const user = useContext(UserContext);
12   return <p>User: {user}</p>;
13 }

```

Interview Tip: Say, “Global state skips the hassle of passing props through many layers.”

21. Redux Toolkit Example

What is it? A simpler way to use Redux with less code.

Example:

```

1 import { configureStore, createSlice } from '@reduxjs/toolkit';
2 import { Provider, useDispatch, useSelector } from 'react-redux';
3
4 // Create slice
5 const counterSlice = createSlice({
6   name: 'counter',
7   initialState: { count: 0 },
8   reducers: {
9     increment(state) { state.count += 1; },
10    decrement(state) { state.count -= 1; },
11  },
12 });
13 const { increment, decrement } = counterSlice.actions;
14
15 // Create store
16 const store = configureStore({
17   reducer: { counter: counterSlice.reducer },
18 });
19
20 // Component
21 function Counter() {
22   const count = useSelector(state => state.counter.count);
23   const dispatch = useDispatch();
24   return (
25     <div>
26       <p>Count: {count}</p>
27       <button onClick={() => dispatch(increment())}>+</button>
28       <button onClick={() => dispatch(decrement())}>-</button>
29     </div>
30   );
31 }
32
33 // App
34 function App() {
35   return (
36     <Provider store={store}>
37       <Counter />
38     </Provider>
39   );
40 }

```

Interview Tip: Explain, “Redux Toolkit is like Redux with training wheels—less boilerplate.”

22. Interview Tips

- **Freshers:** Focus on `useState`, `useEffect`, `props`, and `Context`.
- **Pros:** Know `useMemo`, `useCallback`, `Redux`, and optimization.
- **Explain with Analogies:** Use LEGO for components, drafts for Virtual DOM.
- **Show Examples:** Reference your projects or the code above.

React Docs for more details.